VENTEK AUTONOMOUS ROBOTIC WELDING PROJECT

Team 3 — Multimodal AI & Predictive Analytics

Integrated Final Technical Report

Document version	1.0 — submission draft for integration review
Date	18 October 2026
Prepared for	Project review; integration with Teams 1, 2, 4, 5 and 6
Baseline target	25 October 2026 (see Section 12 for the schedule note)
Interface status	PROPOSED — pending approval by the integration teams
Evidence status	Design and validation plan. No experiments run; all example values are mock data.

Important

Team 3 outputs are advisory. They do not command robot motion and do not replace independent safety interlocks. Unvalidated predictions are published as null until a model, target, labels, units and held-out validation exist.

Contents

2. Executive summary3

3. Problem statement, objective, scope and team responsibilities3
- 3.1 Problem statement3
- 3.2 Objectives4
- 3.3 Scope4
- 3.4 Responsibilities of all six teams [REQ]4
- 3.5 Overall Team 3 processing pipeline5

4. Sensor inventory and multimodal synchronization5
- 4.1 Sensor and data inventory5
- 4.2 Timestamp definitions6
- 4.3 Synchronization and window construction6
- 4.4 Freshness, missing and delayed data7
- 4.5 Data-quality flags7
- 4.6 Data access, privacy and storage7

5. Unified input interface contract: /cell/state7
- 5.1 Topic, producer, consumer and transport8
- 5.2 Example cell_state.json (mock values)8
- 5.3 Field definitions9

6. Phase 1 — Classical ML: features, baseline, Isolation Forest, LightGBM/XGBoost, SHAP10
- 6.1 First task and evaluation reference10
- 6.2 Feature engineering10
- 6.3 Rule and statistical baseline11
- 6.4 Isolation Forest (unsupervised anomaly detection)12
- 6.5 LightGBM and XGBoost (supervised, conditional)12
- 6.6 SHAP explainability13

7. Phase 2 — Advanced deep learning and multimodal fusion (candidates)13
- 7.1 Upgrade gate13
- 7.2 Dual-branch multimodal architecture14
- 7.3 Channel availability and alignment14
- 7.4 Spatial branch and targets15
- 7.5 Variational autoencoder (VAE)15
- 7.6 Evaluation protocol for Phase 215

8. Unified output interface contract: /cell/process_feedback15
- 8.1 Topic, producer and consumers15
- 8.2 Example process_feedback.json (mock values)16
- 8.3 Field definitions, enumerations and sign convention18

9. Inference optimization, deployment and fallback19
- 9.1 The timing requirement: four different quantities19
- 9.2 Candidate deployment stack20
- 9.3 Progressive testing ladder21
- 9.4 Failure handling and fallback21

10. Validation methodology and measurable evaluation metrics22
- 10.1 Validation matrix22
- 10.2 Splits, leakage and statistics22

11. Risks, dependencies and unresolved cross-team decisions23

12. Implementation roadmap and definition of done23
- 12.1 Definition of done (baseline)24

13. References and source basis25
- 13.1 References25
- 13.2 Source basis and claim classes25
- 13.3 Glossary26

2. Executive summary

Team 3 turns synchronized welding-cell data into timestamped, advisory process indicators for Team 4 (adaptive control) and Team 1 (cognitive autonomy). This report defines **what Team 3 will build first, how it will be measured, and what must be agreed with other teams before anything is deployed**. It is a design and validation plan, not a results report: **no experiments have been run, and every numeric value in the JSON examples is mock data.**

- **Scope.** Data ingestion and synchronization, preprocessing, feature extraction, anomaly detection, predictive modelling, model evaluation and process feedback. Team 3 does not command motion, generate trajectories, own safety interlocks or make cell-level decisions.
- **Phase 1 (build now).** Time-domain features over a 50 ms window of confirmed streams, a rule/statistical baseline, then Isolation Forest for unsupervised anomaly detection. LightGBM (with XGBoost as an optional benchmark) and SHAP are used **only if** reliable labels exist. An anomaly is a statistically unusual window, not a confirmed defective weld.
- **Phase 2 (conditional).** A dual-branch CNN-GRU + spatial-MLP model and a VAE are candidates, evaluated against Phase 1 under identical splits. They are kept only if they show a meaningful measured benefit that justifies their cost. The proposed 1,000+ synchronized-log checkpoint is a trigger to assess readiness, not proof of sufficiency.
- **Timing.** A 50 ms window, model-only inference time, end-to-end analytics latency and sensor-to-control reaction time are four different quantities. The sub-10 ms target must be defined precisely before it can be tested; a 50 ms window by itself cannot guarantee a sub-10 ms reaction (Section 9.1).
- **Interfaces.** `/cell/state` (input) and `/cell/process_feedback` (output) are **proposed and pending approval by the integration teams**. Unvalidated predictions are published as `null` / `NOT_AVAILABLE`; missing or stale data never becomes a normal-looking prediction.
- **Safety.** All Team 3 outputs are advisory. No ML output carries stop authority, and none replaces independent safety logic.

> **Status labels used throughout this report**
> **[REQ]** project requirement supplied to Team 3 · **[PROP]** proposed design or interface, pending approval · **[ASSUME]** assumption that must be verified · **[REC]** Team 3 engineering recommendation · **[EXP]** statement whose truth requires an experiment (none has been run) · **[FACT]** established algorithm/framework fact with a cited source.

Five decisions needed first (full list in Section 11): (1) which streams and rates actually exist on the cell, incl. temperature and wire feed; (2) the timestamp source and clock synchronization method; (3) the exact meaning of the <10 ms requirement; (4) whether labelled post-weld inspection data exist, and their source of truth; (5) approval of the `/cell/state` and `/cell/process_feedback` schemas, including the path-offset frame and sign convention.

3. Problem statement, objective, scope and team responsibilities

3.1 Problem statement

During automated welding the cell produces several data streams (electrical telemetry, robot kinematics, vision-derived seam geometry) whose individual interpretation is limited. Deviations from normal behaviour, such as an unstable arc, growing tracking error or a seam that differs from the planned path, are easier to detect and characterize when the streams are aligned in time and analyzed together. Today these streams are not combined into a single, versioned, evaluated analytic product that other teams can consume. [ASSUME: no such product exists yet in VENTEK.]

3.2 Objectives

1. Build a reliable, timestamped ingestion and synchronization layer with explicit data-quality and freshness handling.
2. Establish a measurable baseline (rules/statistics) and an unsupervised anomaly detector (Isolation Forest) on confirmed streams.
3. Define, and only then train, supervised predictors when a clear target and reliable labels exist.
4. Publish one unified advisory payload on `/cell/process_feedback` with model, version, quality and uncertainty metadata.
5. Evaluate advanced multimodal models (Phase 2) honestly against the Phase 1 baseline before adopting them.
6. Prepare the models for edge deployment and measure runtime on the target hardware before making any timing claim.

3.3 Scope

Table 1 — *Team 3 scope boundaries*

In scope (Team 3)	Out of scope (other teams / safety systems)
Ingest, align and quality-check `/cell/state` data Feature extraction and rule/statistical baseline Anomaly detection; supervised models if labels exist Model evaluation, versioning and explainability (SHAP) Publishing `/cell/process_feedback` (advisory) Logging and replay datasets for analysis	Commanding robot motion or fixtures (Teams 2, 6) Trajectory generation, IK and collision checks (Team 6) Path correction and parameter control laws (Team 4) Scene reconstruction and joint detection (Team 5) Cell-level decisions and sequencing (Team 1) Safety interlocks, emergency stop and PLC logic (independent safety systems)

3.4 Responsibilities of all six teams [REQ]

Table 2 — *Division of responsibilities and Team 3 interfaces*

Team	Responsibility	Output	Relation to Team 3
1 — Cognitive Autonomous Welding	High-level decisions, autonomous sequencing, supervisory state	`decision.json`	Consumes optional supervisory quality/prediction information
2 — Cell Orchestration	Robot, fixture, clamp, PLC and ROS 2 coordination; execution state and recovery	`cell_state.json`	Primary producer of `/cell/state` input
3 — Multimodal AI & Predictive Analytics	Ingestion/synchronization, preprocessing, features, anomaly detection, predictive modelling, evaluation, process feedback	`process_feedback.json`	This report
4 — Adaptive Welding & Real-Time Control	Deviation correction, adaptive control, parameter adjustments, replanning triggers	`correction.json`	Consumes optional advisory information; remains the control owner
5 — Computer Vision & Weld Intelligence	Scanning, 3D reconstruction, segmentation, weld-joint geometry	`scene.json`	Source of seam geometry (20 Hz) and joint context
6 — Path Planning, IK & Robot Motion	Kinematics, reachability, collision checks, trajectory generation	`trajectory.json`	Indirect: planned path used as reference for deviation features [ASSUME]

The team overview sheets provided with the project brief show Team 3 working with the tools ROS 2, Python, PyTorch, OpenCV, NumPy, Pandas, scikit-learn, Open3D, MLflow, Docker, TensorBoard and Matplotlib; this report is consistent with that list. Those sheets also contain illustrative outputs (for example a quality score and predicted strength). **Such values are ideation examples and are not validated capabilities**; in this report they stay `null` until a model, target, labels, units and validation exist.

3.5 Overall Team 3 processing pipeline

Figure 1 shows the data path from the proposed input to the proposed output. The independent safety logic of Team 2/PLC sits outside this pipeline and is not affected by it.

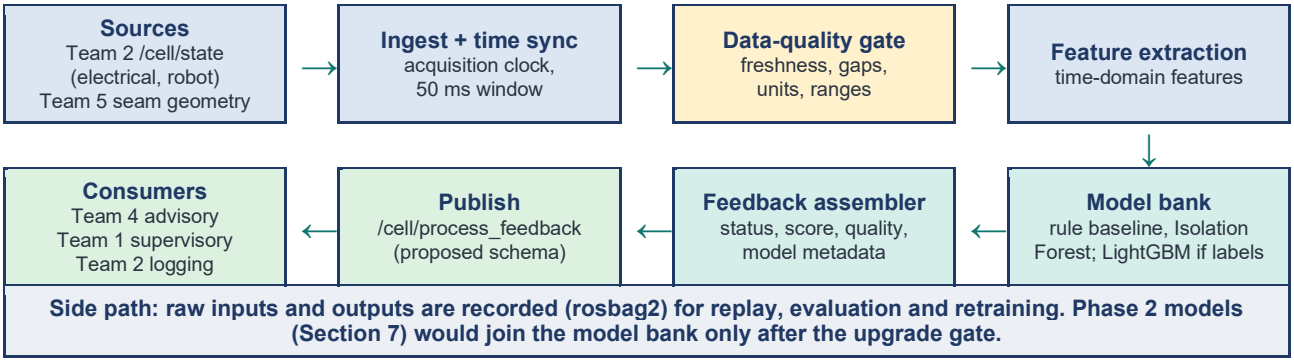


Figure 1 — *Overall Team 3 processing pipeline (proposed).*

4. Sensor inventory and multimodal synchronization

4.1 Sensor and data inventory

Availability describes the **current knowledge state**, not hardware verified by Team 3. “Proposed” means the signal appears in the proposed interface; it still has to be confirmed on the physical cell.

Table 3 — *Signal inventory with availability status*

Signal	Source	Rate	Unit	Availability status	Notes
Welding voltage	Welding power source / electrical telemetry (T2)	1,000 Hz [REQ]	V	**Proposed**; hardware access to confirm	Phase 1 core signal
Welding current	as above	1,000 Hz [REQ]	A	**Proposed**; hardware access to confirm	Phase 1 core signal
Robot joint positions	Robot driver via ROS 2 (T2)	100 Hz [REQ]	rad	**Proposed**	Joint-space context
TCP pose	Robot driver / forward kinematics	100 Hz [REQ]	m; quaternion `[x, y, z, w]`	**Proposed**	Frame to be agreed (Section 5)
TCP linear velocity	Robot driver	100 Hz [REQ]	mm/s	**Proposed**	Mixed-unit exception flagged
Tracking error	Robot controller / planned vs actual	100 Hz [REQ]	mm	**Proposed**	Definition of reference to confirm
Seam geometry (joint ID/type, start/end, normal, root gap)	RGB-D vision pipeline (T5); camera model not confirmed	20 Hz [REQ]	m, unit vector, mm	**Proposed**	Held between frames; age rule in 4.4
Process temperature	Unknown sensor	**Not confirmed**	°C	**Unconfirmed — optional**	Must not be assumed at 1 kHz
Wire-feed speed	Unknown sensor / power source	**Not confirmed**	to confirm	**Unconfirmed — optional**	Unit and rate to confirm
Gas flow	Unknown sensor	**Not confirmed**	to confirm	**Unconfirmed — optional**	
Arc signal, vibration/IMU	Mentioned in team overview sheets	—	—	**Future phase**	No data path defined yet
Weld-quality labels (visual / NDT / destructive)	Post-weld inspection	per weld or segment	class or value	**Unconfirmed**	Needed for any supervised model

4.2 Timestamp definitions

A single field named `timestamp_us` is ambiguous. Team 3 proposes to keep `timestamp_us` as the project-wide field name [REQ] but to give it one precise meaning per message and to name every other time separately [PROP].

Table 4 — *Proposed timestamp semantics (all POSIX epoch microseconds, integer)*

Field	Message	Meaning	Clock
`timestamp_us`	`/cell/state`	**Sample time** of the newest electrical sample in the message	Acquisition clock
`first_sample_timestamp_us`	`/cell/state`	Sample time of the first electrical sample; sample i is at `first + i × sample_period_us`	Acquisition clock
`robot.timestamp_us`, `vision.timestamp_us`	`/cell/state`	Sample time of that stream's latest sample (vision: frame capture time)	Acquisition clock
`publish_timestamp_us`	`/cell/state`	Producer dispatch time (diagnostics and latency only)	Producer ROS clock
`timestamp_us`	`/cell/process_feedback`	**Window closure**: sample time of the newest sample used in this inference	Copied from input
`window_start_timestamp_us`	`/cell/process_feedback`	Sample time of the oldest sample used	Copied from input
`dispatch_timestamp_us`	`/cell/process_feedback`	Time Team 3 published the result (inference dispatch)	Team 3 ROS clock

Unresolved: which physical clock stamps samples (hardware timestamp at the power-source/DAQ interface, PTP, NTP or ROS time), the expected offset between the robot, electrical and camera clocks, and whether `sample_period_us` is truly uniform. The 1 kHz stream is assumed uniform with `sample_period_us = 1000`; this must be checked against measured jitter before it is relied on [ASSUME].

4.3 Synchronization and window construction

1. **Receive** chunks from `/cell/state` and check `sequence_id` continuity (gaps indicate dropped messages).
2. **Validate** schema, units and ranges; reject or flag chunks that fail (Section 4.5).
3. **Align on the acquisition clock.** The electrical stream defines the master time axis. Robot and vision samples are attached by time, never by arrival order.
4. **Build the window.** A 50 ms window at 1,000 Hz contains **50 electrical samples** [REQ]. The robot stream (100 Hz) contributes about **5 samples** and the vision stream (20 Hz, one frame per 50 ms) at most **one new frame**, so vision features are held values whose age is tracked explicitly.
5. **Apply the stride.** The stride sets the output rate and the overlap between consecutive windows. A 10 ms stride would give 100 outputs/s with 80% overlap. The value is **unresolved** [PROP]; overlap matters for validation (Section 10.2).
6. **Compute quality flags**, then features, then inference. A window with insufficient valid data produces `UNKNOWN`, not a prediction.

4.4 Freshness, missing and delayed data

Forward-fill (zero-order hold) is allowed only with a per-stream freshness limit, and every held value is flagged. The limits below are **starting values to be tuned from measured jitter**, not measured facts [PROP].

Table 5 — *Proposed per-stream freshness and completeness rules*

Stream	Nominal period	Hold allowed?	Proposed maximum age / gap	If exceeded
Electrical (1 kHz)	1 ms	No (no fill of electrical samples)	Window needs ≥ 90% of 50 samples; no gap > 5 ms [PROP]	Window `DEGRADED`; below limit → `INVALID`, anomaly `UNKNOWN`

Stream	Nominal period	Hold allowed?	Proposed maximum age / gap	If exceeded
Robot (100 Hz)	10 ms	Yes, flagged `held_value`	Newest sample age ≤ 30 ms (3 periods) [PROP]	Robot features masked; flag `STALE`; models that need them fall back
Vision (20 Hz)	50 ms	Yes, flagged `held_value`	Frame age ≤ 150 ms (3 periods) [PROP]	Vision features masked; `STALE`; Phase 2 spatial branch disabled
Optional (temp., wire feed, gas)	unknown	Not until rate known	To be defined after rate confirmation	Treated as absent; never imputed silently

4.5 Data-quality flags

Each window carries an overall flag (`OK`, `DEGRADED`, `INVALID`) and per-stream status (`OK`, `DEGRADED`, `STALE`, `MISSING`, `INVALID`). Triggers include: sequence gaps, non-monotonic timestamps, clock-offset estimate above limit, values outside physical range (for example negative current), constant (stuck-at) signals, units not matching the schema, and `valid: false` from the producer. Any `INVALID` window yields anomaly status `UNKNOWN`. Quality flags are published so consumers can distinguish “normal” from “not assessed”.

4.6 Data access, privacy and storage

- **Access needed:** read access to the electrical telemetry interface, ROS 2 topics, rosbag2 recordings, and any post-weld inspection records; write access to a shared dataset store and experiment tracking (e.g. MLflow).
- **Sensitivity:** logs can reveal customer part geometry, weld procedures and process parameters. Store under project access control; do not copy logs to personal devices or external services. Retention period and ownership are **unresolved**.
- **Dataset hygiene:** record every run with run ID, operator, material, thickness, joint type, process parameters and clock source, so splits can be made by run/session and operating conditions (Section 10.2).
- **Format [REC]:** rosbag2 for raw capture; columnar files (e.g. Parquet) for derived window/feature tables with schema version stored alongside.

5. Unified input interface contract: /cell/state

> **Proposed — pending approval by the integration teams**
> Topic name, message rate and field names below are **proposals**. Team 3 will not treat them as an approved contract until Teams 1, 2, 4 and 5 agree. The JSON is the logical schema; the ROS 2 transport encoding (typed message vs JSON string) is an open decision.

5.1 Topic, producer, consumer and transport

Table 6 — *Input interface summary*

Item	Proposal	Status
Topic	`/cell/state`	[REQ] name; contents [PROP]
Producer	Team 2 (aggregates robot, electrical, PLC; relays Team 5 seam geometry). Ownership of the vision relay is unresolved.	[PROP]
Consumer	Team 3 (also logged by Team 2 for replay)	[PROP]
Message granularity	Chunks of newly acquired data (e.g. 10 ms = 10 electrical samples, 1 robot sample, latest vision frame). Team 3 assembles the sliding 50 ms window. Alternative: producer sends full 50 ms windows.	[PROP] open decision
Encoding	Typed ROS 2 message mirroring this schema is preferred for efficiency; JSON strings are acceptable for early integration	[REC]

Item	Proposal	Status
QoS	Compatibility between publisher and subscriber reliability/durability policies must be checked; best-effort vs reliable to be decided from loopback tests	[FACT] ROS 2 QoS rules; choice [EXP]
Units	SI where possible. Exceptions inherited from the brief: tcp_speed in mm/s, tracking error and root gap in mm. Flagged for agreement.	[REQ]/[PROP]
Quaternion order	`[x, y, z, w]`	[PROP] per project guide
Coordinate frame	Robot base frame `base_link` for TCP and seam coordinates; to be confirmed with Teams 5 and 6	[PROP]

5.2 Example cell_state.json (mock values)

All values are illustrative **mock data**, not measurements. The example shows a 10 ms chunk; `optional_process` fields stay `null` because those sensors are unconfirmed.


```
{
  "schema_version": "0.1-proposed",
  "data_origin": "MOCK_EXAMPLE_NOT_MEASURED",
  "sequence_id": 482913,
  "timestamp_us": 1792317600129000,
  "publish_timestamp_us": 1792317600131500,
  "clock": {
    "source": "UNCONFIRMED",
    "offset_estimate_us": null
  },
  "cell": {
    "mode": "WELDING",
    "active_joint_id": "J2",
    "arc_on": true
  },
  "electrical": {
    "rate_hz": 1000,
    "first_sample_timestamp_us": 1792317600120000,
    "sample_period_us": 1000,
    "voltage_v": [23.9, 24.1, 24, 23.8, 24.2, 24, 23.9, 24.1, 24, 23.9],
    "current_a": [198.2, 201.5, 199.8, 197.4, 202.1, 200.3, 198.9, 200.7, 199.5, 198.8],
    "dropped_samples": 0,
    "valid": true
  },
  "robot": {
    "rate_hz": 100,
    "timestamp_us": 1792317600125000,
    "frame_id": "base_link",
    "joint_positions_rad": [0.12, -1.05, 1.31, 0.02, 1.38, -0.31],
    "tcp_position_m": [0.412, -0.087, 0.265],
    "tcp_orientation_xyzw": [0, 0.7071, 0, 0.7071],
    "tcp_speed_mm_s": 8,
    "tracking_error_mm": 0.35,
    "valid": true
  },
  "vision": {
    "rate_hz": 20,
    "timestamp_us": 1792317600090000,
    "frame_id": "base_link",
    "joint_id": "J2",
    "joint_type": "fillet",
    "seam_start_m": [0.4, -0.09, 0.26],
    "seam_end_m": [0.52, -0.09, 0.26],
    "surface_normal": [0, 0, 1],
    "root_gap_mm": 0.4,
    "confidence": 0.91,
    "valid": true
  },
  "optional_process": {
    "temperature_c": {
      "value": null,
      "status": "UNCONFIRMED"
    },
    "wire_feed_speed_mm_s": {
      "value": null,
      "status": "UNCONFIRMED"
    },
    "gas_flow_l_min": {
      "value": null,
      "status": "UNCONFIRMED"
    }
  }
}
```

Listing 1 — *Example `cell_state.json` (proposed schema, mock data, JSON validated programmatically).*

5.3 Field definitions

Table 7 — *`cell_state.json` field definitions (producer: Team 2, consumer: Team 3)*

Field	Type, unit	Req.	Meaning / timestamp semantics	If unavailable
`schema_version`	string	Req	Schema version, e.g. `0.1-proposed`	Reject message
`data_origin`	string	Opt	Present only in mock/replay data; must be absent in live data	Treated as live
`sequence_id`	integer	Req	Monotonic counter per producer; gaps reveal dropped messages	Reject
`timestamp_us`	integer, µs epoch	Req	Sample time of newest electrical sample (acquisition clock)	Reject
`publish_timestamp_us`	integer, µs	Req	Producer dispatch time; diagnostics only	Latency not computed
`clock.source`, `offset_estimate_us`	enum, µs	Opt	Clock type (PTP, NTP, ROS_TIME, UNCONFIRMED) and estimated offset to master	`UNCONFIRMED`; window flagged
`cell.mode`	enum (Team 2 defines)	Opt	Execution state, e.g. WELDING; used to select arc-on windows	Windows tagged `context_unknown`
`cell.active_joint_id`	string or null	Opt	Joint being welded	`null`; output joint ID null
`cell.arc_on`	boolean or null	Opt	Arc state from power source/PLC	`null`; arc-on masking disabled
`electrical.*`	arrays, V and A	Req	Voltage/current samples; time per sample = first + i × period	Window `INVALID`
`electrical.dropped_samples`	integer	Opt	Producer-reported missing samples in chunk	Inferred from timestamps
`robot.*`	rad, m, `[x,y,z,w]`, mm/s, mm	Req*	Latest robot sample in `frame_id`; *required for tracking-error features	Robot features masked
`vision.*`	m, unit vector, mm, 0–1	Opt	Latest seam geometry; `confidence` is the producer's own confidence, not calibrated by Team 3	Spatial features masked
`optional_process.*`	value + status	Opt	Unconfirmed sensors; `value` null while `status` is `UNCONFIRMED`	Not used

6. Phase 1 — Classical ML: features, baseline, Isolation Forest, LightGBM/XGBoost, SHAP

6.1 First task and evaluation reference

First task [PROP]: window-level anomaly detection on arc-on windows built from the confirmed streams (electrical, robot tracking error, optionally vision-derived gap). The output is an anomaly status and score, **not** a defect diagnosis.

Table 8 — *Reference data for evaluating the first task*

Reference	What it can show	What it cannot show	Availability
Injected synthetic anomalies (spike, dropout, drift, stuck-at, noise burst, step in tracking error)	Software correctness, detection delay, false-alarm behaviour under known events	Real-world defect-detection accuracy	Available (generated by Team 3)
Operator/expert annotations of unstable-arc or abnormal events on logs	Agreement of detector with expert judgement	Weld metallurgical quality	[ASSUME] to be arranged
Post-weld inspection (visual/NDT/destructive) per weld or segment	Relationship between anomalies and inspected quality	—	**Unconfirmed**

6.2 Feature engineering

Features are computed per window of N samples x_1 … x_N with sample period Δt (N = 50, Δt = 1 ms for the electrical stream). Start with simple time-domain features on confirmed streams [REC].

Table 9 — *Feature definitions*

Feature	Formula	Interpretation / caution
Mean μ	μ = (1/N) Σ x_k	Operating level (e.g. mean current). Slow drifts shift it.
Standard deviation σ	σ = √[(1/N) Σ (x_k − μ)²] (population form, ddof = 0; fixed for training and inference)	Short-term variability. Rises with arc instability, but also with normal droplet transfer; baseline levels are process-dependent.
Minimum, maximum	min_k x_k, max_k x_k	Extremes; sensitive to single-sample spikes and glitches.
Range R	R = max − min	Peak-to-peak swing in the window.
Root mean square	RMS = √[(1/N) Σ x_k²]	Signal energy. RMS² = μ² + σ² (population form), so RMS partly duplicates mean and std.
Mean rate of change	$\bar{r}$ = (x_N − x_1) / ((N − 1) Δt)	Net trend over the window, in units/s.
Maximum rate of change	r_max = max_k |x_{k+1} − x_k| / Δt	Fastest step; amplifies measurement noise, so compare against a noise floor measured on real data.
Mean power (derived, candidate)	$\bar{P}$ = (1/N) Σ v_k · i_k	Electrical power proxy; valid only if voltage and current samples are time-aligned [ASSUME].

Table 10 — *Initial candidate feature set (counts)*

Group	Signals	Features per signal	Count
Electrical statistics	voltage, current	mean, std, min, max, range, RMS, mean rate, max rate (8)	16
Electrical derived	voltage × current	mean power (1)	1
Robot	tracking error; TCP speed	tracking error: mean, max |e| (2); speed: mean, std (2)	4
Vision (held values)	root gap, confidence, frame age	last value (1 each)	3
Total candidate features			**24**

Feature code is unit-tested against hand-calculated values and replayed logs before any model is trained (Section 10). Any smoothing used before rate-of-change features must be causal, so no future samples leak into a window.

6.3 Rule and statistical baseline

The baseline is the first reference point. A more complex model is only worth its cost if it detects more real events than the baseline at the **same false-alarm budget** on held-out runs [REC]. Engineering limits come from the welding procedure specification (WPS) and robot specifications; **Team 3 does not invent numeric limits** [ASSUME: WPS values will be provided].

Table 11 — *Baseline rules*

ID	Rule	Source of limit	Flag
R1	Window mean or extremes of current outside [I_min, I_max]	WPS (TBD)	`CURRENT_LIMIT`
R2	Window mean or extremes of voltage outside [V_min, V_max]	WPS (TBD)	`VOLTAGE_LIMIT`
R3	Rolling z-score of window mean or std vs a reference built from the preceding stable part of the same run exceeds k	k tuned on validation runs	`LEVEL_SHIFT`, `VARIABILITY`
R4	Max rate of change above noise-floor multiple	Measured noise floor (EXP)	`RATE_LIMIT`
R5	Mean or max tracking error above limit	Team 4 / robot spec (TBD)	`TRACKING_ERROR`
R6	Data-quality gate failed	Section 4.4/4.5	`DATA_INVALID`

6.4 Isolation Forest (unsupervised anomaly detection)

How it works [FACT]. Isolation Forest builds an ensemble of random trees. Each tree repeatedly picks a random feature and a random split value; points that are few and different are isolated in fewer splits, so they have shorter average path lengths (Liu, Ting and Zhou, 2008). The paper's score is s(x, n) = 2^(−E[h(x)] / c(n)), which approaches 1 for anomalies. In scikit-learn, `score_samples` returns the opposite of this score, so lower values are more abnormal; Team 3 will publish `score = -score_samples(x)` so that **higher means more anomalous** [REC]. scikit-learn defaults include 100 trees and `max_samples = "auto"`, i.e. min(256, n_samples).

Training data. Normal windows from complete runs that cover the intended operating conditions (materials, thicknesses, joint types, travel speeds, wire/gas settings). The detector only learns what "normal" looked like in the training runs; unseen but healthy conditions will score as anomalous.

Threshold selection [REC]. Do not rely on a default contamination. Fit on training runs, score separate validation runs, and choose a threshold from either (a) a target false-alarm rate per hour of arc-on time on normal validation runs or (b) the best trade-off on annotated or injected events. Freeze the model and threshold, then evaluate once on held-out test runs. Publish `threshold_validated = true` only after this procedure.

What is reported: precision, recall and F1 at event level, confusion matrix, false alarms per run and per hour, missed events and detection delay (time from event onset to first flagged window).

> **An anomaly is not automatically a defective weld**
> A flagged window means "unusual relative to the training data". Causes include genuine process instability, but also a new workpiece or material, a changed parameter set, arc start/end transients, a sensor glitch or a communication gap. Downstream consumers must treat the flag as a prompt for review or monitoring, never as proof of a defect.

Illustrative training/scoring sketch (not executed; for orientation only):

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(n_estimators=100, max_samples=256, random_state=0)
model.fit(X_train_normal)               # windows from TRAIN runs only
score = -model.score_samples(X)         # higher = more anomalous (paper-style)
flag  = score > threshold               # threshold chosen on VALIDATION runs only
```

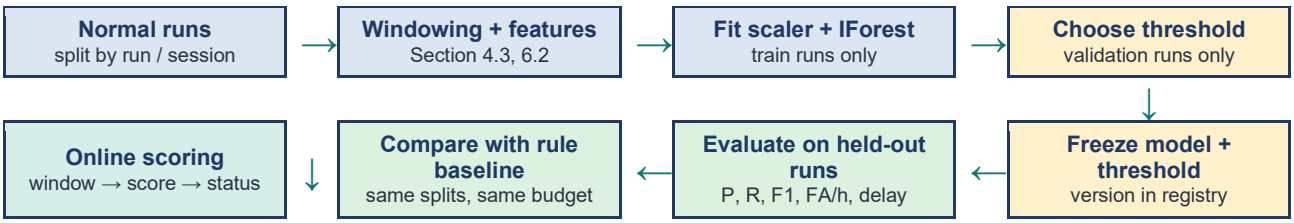


Figure 2 — *Phase 1 baseline and Isolation Forest workflow. Training, threshold choice and evaluation use disjoint runs.*

6.5 LightGBM and XGBoost (supervised, conditional)

What they are [FACT]. Both are gradient-boosted decision-tree methods for tabular features: trees are added one after another, each fitted to the errors of the ensemble so far. LightGBM (Ke et al., 2017) speeds up training with Gradient-based One-Side Sampling and Exclusive Feature Bundling; XGBoost (Chen and Guestrin, 2016) is a scalable tree-boosting system with sparsity-aware split finding. **LightGBM is the primary candidate; XGBoost is an optional benchmark.**

Entry gate [REC]. Train only when (1) a target is defined with a documented source of truth, (2) labels are reliable and linked to windows or segments, (3) there are enough independent runs per class to hold out complete runs, and (4) the Phase 1 baseline has a measurable limitation to improve on. Otherwise this step is skipped and recorded as "not started: no labels".

Table 12 — *Candidate supervised targets (none validated; label availability unconfirmed)*

Target	Type	Candidate label source	Metrics	Status
Arc-instability event	binary / event	Expert annotation of logs	P, R, F1, PR-AUC, delay	[ASSUME]
Segment quality pass/fail	binary	Visual inspection per segment	P, R, F1, confusion matrix	Unconfirmed
Defect class (porosity, undercut, …)	multi-class	NDT or detailed inspection	Per-class P/R, macro-F1	Unconfirmed
Path deviation	regression (mm)	Seam-tracking reference (Teams 4/5)	MAE, RMSE	Unconfirmed
Weld strength	regression	Destructive tests	MAE, RMSE, calibration	Unlikely near-term

Class imbalance is expected; use class weights or resampling **inside training folds only**, report precision–recall metrics rather than accuracy alone, and report calibration (e.g. reliability curves) before publishing any probability. A published `defect_probability_calibrated` requires demonstrated calibration on held-out runs.

6.6 SHAP explainability

What it provides [FACT]. SHAP assigns each feature a contribution to an individual prediction, based on Shapley values (Lundberg and Lee, 2017). For tree ensembles, TreeExplainer computes exact Shapley values in polynomial time and supports local, interaction and global analyses (Lundberg et al., 2020). It is applicable to LightGBM and XGBoost; Isolation Forest is outside this plan's SHAP scope unless separately verified.

- **It explains model behaviour, not physical causation.** A large SHAP value for "current std" means the model relied on it, not that current variability caused a defect. Correlated features share credit unpredictably.
- **Use after evaluation [REC].** Run SHAP on held-out data to sanity-check that the model uses plausible features and not artefacts (run ID, position in weld, clock offsets). It is not a substitute for validation.
- **Offline only.** SHAP is a diagnostic and reporting tool; it is not part of the real-time inference path. Summaries are stored with each model version.

7. Phase 2 — Advanced deep learning and multimodal fusion (candidates)

Everything in this section is a **candidate to evaluate**, not a guaranteed upgrade. None of the architectures or dimensions below has been trained or tested.

7.1 Upgrade gate

The member proposals mention 1,000+ synchronized weld logs as a Phase 2 milestone. Team 3 treats this as a **checkpoint to trigger a readiness review**, not a guarantee of sufficiency [PROP]. Readiness depends on:

- **Run diversity:** number of distinct runs, workpieces, materials, joint types and sessions, not just the number of logged windows.
- **Label quality:** a defined source of truth and verified linkage between labels and data.
- **Coverage:** operating conditions the deployed system will meet, including rare ones.
- **Baseline limitations:** an evidenced case where Phase 1 fails (e.g. misses events that need multi-signal or spatial context).
- **Leakage-safe validation:** splits by complete run, workpiece or session (Section 10.2).

7.2 Dual-branch multimodal architecture

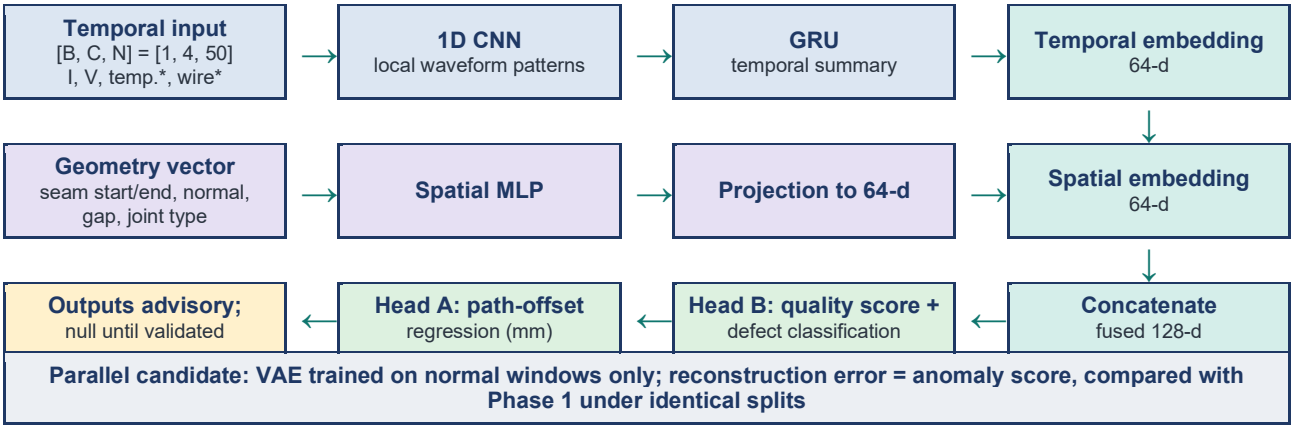


Figure 3 — *Phase 2 multimodal fusion candidate. *Temperature and wire-feed channels are unconfirmed and are not assumed available at 1 kHz.*

Table 13 — *Candidate components, shapes and training needs (illustrative hyper-parameters are not tuned)*

Component	Role	Shape / size (candidate)	Training target needed
Temporal input	Window of process signals	[B, C, N] = [1, 4, 50]; C = current, voltage, process temperature, wire-feed speed	—
1D CNN	Extracts local waveform patterns (spikes, ripple, short-circuit-like events) along time	e.g. 2 conv layers with ReLU; kernel sizes and channels to be tuned	Trained end-to-end with heads
GRU	Summarizes the CNN sequence into a state vector; handles ordering and short memory	Final hidden state → 64-d	End-to-end
Spatial MLP	Encodes weld-joint geometry	Input dimension to be defined with Team 5; output 64-d	End-to-end
Fusion	Concatenation of branch embeddings	64 + 64 = 128-d	—
Head A: path offset	Regress an advisory seam/path offset	Output in mm, frame and sign per Section 8.3	Measured offset from an independent reference (e.g. Team 5/4 seam tracking), not available yet
Head B: quality score + defect class	Score and classify	Score in [0, 1]; classes TBD	Labelled post-weld inspection (unconfirmed)

7.3 Channel availability and alignment

Only electrical telemetry has an assigned 1 kHz rate. **Process temperature and wire-feed speed must not be assumed to be 1 kHz signals.** Before the four-channel input can be used, the team must confirm the real rate, unit and latency of each. Options to compare [REC]:

- **Electrical-only start:** shape [1, 2, 50] (current, voltage) — the only configuration supported by confirmed data.
- **Resample with a mask:** hold slower signals with a per-sample freshness flag and a mask channel so the network can ignore stale values; hold limits follow Section 4.4.
- **Multi-rate encoders:** separate encoders per rate fused at the embedding level, if slow signals prove informative.

7.4 Spatial branch and targets

The spatial MLP consumes weld-joint geometry from Team 5 (seam start/end, surface normal, root gap, joint type encoded categorically). Geometry changes slowly relative to the electrical signal, so the branch input is a held value with age tracked; if it is stale or low-confidence the branch is disabled and the model falls back (Section 9.4). Head A needs a measured offset reference in a defined frame and unit; Head B needs

inspection labels. Without them, the heads are not trained and their outputs stay `null`. A multi-task loss (regression + classification) needs weighting that is chosen on validation runs.

7.5 Variational autoencoder (VAE)

How it works [FACT]. A VAE learns a compressed latent representation of its training data: an encoder outputs the mean and variance of a latent distribution, a sample is decoded back to the input, and training balances reconstruction accuracy against a KL-divergence regulariser (Kingma and Welling, 2013). Trained **only on normal windows**, it should reconstruct normal patterns well; the **reconstruction error** then serves as an anomaly score.

- **Same test conditions:** identical windows, feature scaling, run-level splits, threshold procedure and metrics as the rule baseline and Isolation Forest, so the comparison is fair.
- **Caution [FACT-adjacent]:** reconstruction error depends on input scaling and can be small for some unfamiliar inputs; it needs the same out-of-distribution checks as any other detector.
- **Retention rule:** the VAE is kept only if it detects meaningfully more real or injected events, or detects them earlier, at the same false-alarm budget as Phase 1 and within the timing budget.

7.6 Evaluation protocol for Phase 2

- **Ablations:** electrical-only vs + robot vs + vision; with and without optional channels; several random seeds.
- **Cost accounting:** parameter count, model-only inference time and memory on the target hardware, compared with Phase 1.
- **Decision rule [PROP]:** adopt a Phase 2 model only if the improvement is meaningful on held-out runs (agreed margin, with uncertainty from run-level resampling) and the timing/resource cost is acceptable; otherwise keep Phase 1.

8. Unified output interface contract: /cell/process_feedback

> **Proposed — pending approval by the integration teams**
> One payload for all Team 3 outputs. Teams 1 and 4 each receive their own optional block inside the same message; there are no separate, incompatible JSON formats. Fields for defect class, strength, calibrated probability and path offsets are `null` until the model, target, labels, units and validation exist.

8.1 Topic, producer and consumers

Table 14 — *Output interface summary*

Item	Proposal
Topic	`/cell/process_feedback` [REQ name; contents PROP]
Producer	Team 3 inference node
Consumers	Team 4 (`advisory_team4` block), Team 1 (`supervisory_team1` block), Team 2 (logging and display). Each consumer ignores blocks it does not use.
Rate	One message per window stride (default proposal 10 ms stride → up to 100 Hz; unresolved)
Authority	**Advisory only.** No field commands motion or stops the cell. No enumerator exists for stop or emergency stop.
Initial deployment mode	`shadow_mode: true`: outputs are logged and compared with other evidence; consumers must not act on them until validated and approved.

8.2 Example process_feedback.json (mock values)

Mock data, not measured results. The anomaly score and threshold are placeholders; latency fields are `null` because **no timing has been measured**.


```
{
  "schema_version": "0.1-proposed",
  "data_origin": "MOCK_EXAMPLE_NOT_MEASURED",
  "sequence_id": 96412,
  "source_sequence_ids": {
    "first": 482909,
    "last": 482913
  },
  "timestamp_us": 1792317600129000,
  "window_start_timestamp_us": 1792317600080000,
  "dispatch_timestamp_us": 1792317600134000,
  "window": {
    "duration_ms": 50,
    "stride_ms": 10,
    "samples_electrical": 50
  },
  "latency": {
    "analytics_latency_us": null,
    "model_inference_us": null,
    "measured": false
  },
  "active_joint_id": "J2",
  "data_quality": {
    "overall": "OK",
    "streams": {
      "electrical": {
        "status": "OK",
        "samples_present": 50,
        "samples_expected": 50,
        "max_gap_us": 1000,
        "held_value": false
      },
      "robot": {
        "status": "OK",
        "age_us": 4000,
        "max_age_us": 30000,
        "held_value": false
      },
      "vision": {
        "status": "OK",
        "age_us": 39000,
        "max_age_us": 150000,
        "held_value": true
      }
    }
  },
  "anomaly": {
    "status": "NORMAL",
    "shadow_mode": true,
    "model_id": "iforest_v0",
    "model_phase": "PHASE_1",
    "model_version": "0.0.0-mock",
    "score": 0.47,
    "threshold": 0.62,
    "threshold_validated": false,
    "rule_baseline": {
      "model_id": "rule_baseline_v0",
      "flags": [],
      "limits_source": "WPS_TBD"
    }
  },
  "advisory_team4": {
    "available": false,
    "status": "NOT_AVAILABLE",
    "suggested_action": "NONE",
    "path_offset_mm": null,
    "offset_frame": "seam_frame",
    "offset_valid": false,
    "confidence": null
  },
  "supervisory_team1": {
    "available": false,
    "status": "NOT_AVAILABLE",
    "recommended_action": "NOT_AVAILABLE",
    "quality_score": null,
    "defect_class": null,
    "defect_probability_calibrated": null,
    "predicted_strength_mpa": null
  }
```

```
}
```

Listing 2 — *Example `process_feedback.json` for a healthy window in shadow mode (proposed schema, mock data).*

Example of the same schema when the data are invalid. The result is `UNKNOWN`, not a normal-looking score. Only fields that differ from the previous example are shown below, inside the full structure:

```
{
  "sequence_id": 96413,
  "data_quality": {
    "overall": "INVALID",
    "streams": {
      "electrical": {
        "status": "MISSING",
        "samples_present": 31,
        "samples_expected": 50,
        "max_gap_us": 19000,
        "held_value": false
      },
      "robot": {
        "status": "OK",
        "age_us": 4000,
        "max_age_us": 30000,
        "held_value": false
      },
      "vision": {
        "status": "OK",
        "age_us": 39000,
        "max_age_us": 150000,
        "held_value": true
      }
    }
  },
  "anomaly": {
    "status": "UNKNOWN",
    "shadow_mode": true,
    "model_id": "iforest_v0",
    "model_phase": "PHASE_1",
    "model_version": "0.0.0-mock",
    "score": null,
    "threshold": 0.62,
    "threshold_validated": false,
    "rule_baseline": {
      "model_id": "rule_baseline_v0",
      "flags": ["DATA_INVALID"],
      "limits_source": "WPS_TBD"
    }
  },
  "...": "all other fields as in the previous example (shown here as a string placeholder; omit in real
messages)"
}
```

Listing 3 — *Differences for an invalid-data window (fragment of the same schema; the `...` key is a documentation placeholder).*

8.3 Field definitions, enumerations and sign convention

Table 15 — *`process_feedback.json` field definitions (producer: Team 3)*

Field	Type, unit	Req.	Meaning / semantics	Consumer	If unavailable
`schema_version`, `data_origin`	string	Req / Opt	Version; `data_origin` only in mock/replay data	All	Reject / treated live
`sequence_id`	integer	Req	Monotonic output counter (heartbeat: gaps = missed outputs)	All	—
`source_sequence_ids`	object of integers	Req	First/last `/cell/state` chunk used	Team 2 (trace)	—
`timestamp_us`	integer, µs epoch	Req	Window closure: sample time of newest sample used	All	—
`window_start_timestamp_us`	integer, µs	Req	Sample time of oldest sample used	All	—
`dispatch_timestamp_us`	integer, µs	Req	Time Team 3 published this message	Team 2 (latency)	—
`window.*`	ms, count	Req	Window duration, stride, electrical samples used	All	—
`latency.*`	integer µs or null	Opt	Measured analytics and model-only latency; `measured` false until benchmarked	Team 2	`null`, `measured: false`
`active_joint_id`	string or null	Opt	Joint ID from input context	Teams 1, 4	`null`
`data_quality.overall`	enum	Req	`OK`, `DEGRADED`, `INVALID`	All	—

Field	Type, unit	Req.	Meaning / semantics	Consumer	If unavailable
`data_quality.streams.*`	status, µs, counts	Req	Per-stream status, age of newest sample at window closure, max allowed age, gaps, held-value flag	All	Status `MISSING`
`anomaly.status`	enum	Req	`NORMAL`, `ANOMALY`, `UNKNOWN` (not assessed)	Teams 1, 4	`UNKNOWN`
`anomaly.shadow_mode`	boolean	Req	True: do not act on this output	All	—
`anomaly.model_id`, `model_phase`, `model_version`	string, enum	Req	Model identity; phase `BASELINE_RULES`, `PHASE_1`, `PHASE_2`	All	—
`anomaly.score`	float or null, unitless	Opt	Paper-style Isolation Forest score; higher = more anomalous. **Not a probability.**	Teams 1, 4	`null` when `UNKNOWN`
`anomaly.threshold`, `threshold_validated`	float or null; boolean	Opt / Req	Decision threshold and whether it passed the validation procedure of Section 6.4	Teams 1, 4	`null`; `false`
`anomaly.rule_baseline.*`	string, string[]	Opt	Baseline model ID and triggered rule flags (R1–R6); `limits_source` records origin of limits	Teams 1, 4	Empty list
`advisory_team4.*`	object	Opt	Block for Team 4. `suggested_action` enum: `NONE`, `MONITOR`, `SUGGEST_SEAM_RECHECK`, `SUGGEST_HUMAN_REVIEW`. Advisory only; Team 4 decides.	Team 4	`available: false`
`advisory_team4.path_offset_mm`	array [x, y, z] mm or null	Opt	Suggested seam offset in `offset_frame`; valid only if `offset_valid`	Team 4	`null`; `offset_valid: false`
`supervisory_team1.*`	object	Opt	Block for Team 1. `recommended_action` enum: `CONTINUE_MONITORING`, `FLAG_FOR_INSPECTION`, `SUGGEST_HUMAN_REVIEW`, `NOT_AVAILABLE`	Team 1	`available: false`
`quality_score`, `defect_class`, `defect_probability_calibrated`, `predicted_strength_mpa`	float, string, float, MPa	Opt	Supervised outputs; remain `null` until target, labels, units and held-out validation exist	Team 1	`null`

Path-offset sign convention (proposed — needs Team 4/5/6 agreement)

`seam_frame`: origin at the nearest point on the planned seam; **+x** along the travel direction (seam start → seam end); **+z** along the surface normal, pointing away from the workpiece; **+y = z × x** (right-handed), which points to the left when looking along travel with the normal "up". A positive `y` offset means the suggested seam position is to the left of the planned path. Units are millimetres (an explicit exception to SI, inherited from the brief). Quaternions elsewhere use `[x, y, z, w]`.

9. Inference optimization, deployment and fallback

9.1 The timing requirement: four different quantities

The architecture documents propose a target of **less than 10 ms** for AI inference or feedback processing. **The exact boundary of this requirement is unconfirmed** and must be agreed before it can be tested [REQ, unresolved]. The proposed 50 ms window does not by itself guarantee a sub-10 ms response, because a decision based on a 50 ms window uses 50 ms of context and several further delays follow.

Table 16 — *Timing quantities that must not be conflated*

Quantity	Definition	Who owns / measures	Status
Window duration	Time span of data used for one inference: 50 ms (50 electrical samples at 1 kHz)	Design parameter (Team 3)	[REQ] proposed initial value
Model-only inference time	Time inside the model call for one prepared feature vector or tensor	Team 3, on target hardware	**Not measured**
End-to-end analytics latency	From window closure (newest sample time) to `/cell/process_feedback` dispatch: synchronization, preprocessing, features, inference, serialization, publish	Team 3 with Team 2 timestamps	**Not measured**
Sensor-to-control reaction time	From a physical event at the sensor to a changed actuator command, including transport and Team 4/Team 2 processing	Teams 2 and 4 jointly	**Not measured; outside Team 3 alone**

Window versus stride. A sliding window produces a new output each stride, so the output rate can be high, but each output summarizes the last 50 ms. How fast an event shows up in a window feature depends on the feature, the event size and the threshold, so detection delay has to be **measured** (Section 10), not inferred from the window length.

> **If the complete control loop must react within 10 ms**
> A 50 ms window-based analytic cannot be the 10 ms control path. Options to be decided with Teams 2 and 4 [REC]: (a) reinterpret the target as model-only or analytics-processing time, with reaction time defined separately; (b) add a short-window path (e.g. a few milliseconds of samples with simple rules) implemented close to the controller or PLC; (c) keep Team 3 as the slower advisory layer. Any fast path that protects the process or people belongs to independent control and safety logic, not to Team 3's ML.

9.2 Candidate deployment stack

- **ONNX export [FACT]:** PyTorch's `torch.onnx` module converts a native `torch.nn.Module` into an ONNX graph. Recurrent layers (GRU) must be tested explicitly: the exported model has to be compared numerically against the PyTorch model on real windows and checked for supported opset, input shapes and batch behaviour [EXP].
- **ONNX Runtime [FACT]:** runs ONNX models through pluggable execution providers; the CPU provider is the default, and accelerators are selected by listing providers when creating the session.
- **TensorRT execution provider [FACT]:** can accelerate supported subgraphs on NVIDIA GPUs; unsupported operators fall back to other providers, and engine build/caching options exist. The team overview sheets list NVIDIA Jetson as a candidate edge platform for other teams; Team 3's target edge hardware is **unconfirmed**.
- **No acceleration is assumed.** Whether ONNX Runtime or TensorRT is faster than native PyTorch or a CPU scikit-learn/LightGBM call for these small inputs can only be established by benchmarks on the target device [EXP]. For Phase 1 tree models, native inference may be sufficient.
- **Model registry and tracking [REC]:** MLflow for versions, parameters, metrics and data-set references; every published message carries `model_id`, `model_phase` and `model_version`.

9.3 Progressive testing ladder

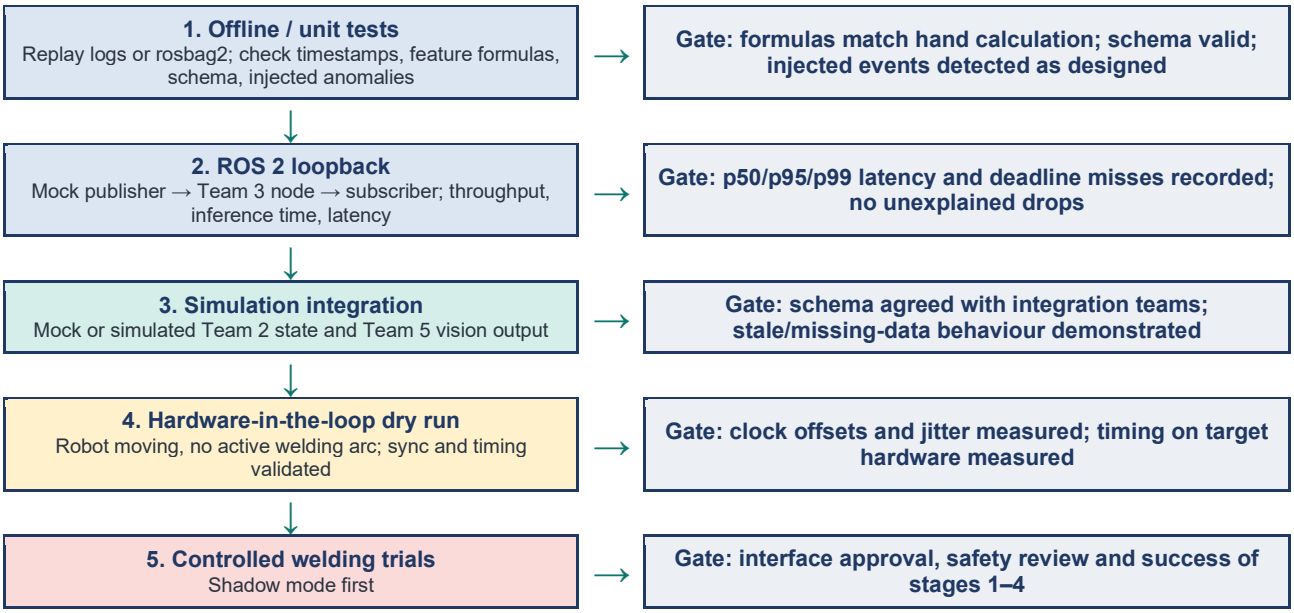


Figure 4 — *Deployment and progressive testing ladder. A stage starts only after the gate of the previous stage is met.*

9.4 Failure handling and fallback

Table 17 — *Failure modes and proposed behaviour*

Situation	Detection	Proposed behaviour
Missing or stale packets	`sequence_id` gaps; sample count; age limits (Section 4.4)	Flag stream; mask dependent features; `DEGRADED`/`INVALID`; anomaly `UNKNOWN` when below completeness limit. Never fill silently.
Network jitter, packet loss	Inter-arrival statistics; `publish_timestamp_us` vs arrival; sequence gaps	Window built by sample time, not arrival; record jitter in logs; adjust QoS after loopback tests [EXP].
Out-of-distribution data	Feature range checks against training statistics; detector score on inputs; unknown operating-condition tag	Report `UNKNOWN` or flag low trust rather than a confident score; log for review and retraining.
Changed workpiece, material or process conditions	Changes in `joint_type`, parameters or run metadata; baseline shift	Re-establish reference (rule R3) and re-validate or retrain thresholds before trusting scores; do not carry thresholds across conditions silently.
Inference deadline miss	Dispatch time vs configured budget; watchdog	Count misses; publish `UNKNOWN` or the previous valid result flagged stale; trigger fallback ladder below.
Team 3 node stops publishing	`sequence_id` and `dispatch_timestamp_us` stop advancing	Consumers (Teams 1, 2, 4) must treat the absence of fresh feedback as “no information”; timeout owned by consumers.

Fallback ladder [PROP, to be tested]: Phase 2 model → Isolation Forest → rule baseline → `UNKNOWN`. The ladder is triggered by deadline misses, invalid inputs for a branch (e.g. stale vision disables the spatial branch) or a failed model health check. **It is not assumed that a lower rung is faster; that must be measured.** AI fallback is not a replacement for independent safety systems, which remain with Team 2, the PLC and the cell’s hardware interlocks.

10. Validation methodology and measurable evaluation metrics

10.1 Validation matrix

Table 18 — *Validation experiments, metrics and evidence (no result exists yet)*

Area	Experiment	Metrics / evidence	Data
Schema and timestamp validation	Validate messages against schema; check monotonic time, sequence continuity, unit ranges; replay logs and compare window boundaries with expected	100% of malformed mock messages rejected; window sample counts exact; formula tests pass	Mock publisher, rosbag2 replay
Rule/statistical baseline	Run R1–R6 on normal and injected-anomaly runs	Event-level precision, recall, F1; false alarms per run and per arc-on hour; detection delay	Normal runs + injected events; annotated real runs if available
Isolation Forest	Train on normal runs; threshold on validation runs; evaluate on held-out runs; compare with baseline	As above + confusion matrix; sensitivity to window/stride/feature set; performance per operating condition	Run-level splits
Supervised prediction (if labels exist)	LightGBM (and XGBoost benchmark) with run-grouped cross-validation, final held-out test	Precision, recall, F1, PR-AUC; MAE/RMSE for regression; calibration metrics if probabilities are claimed; SHAP sanity checks	Labelled inspection data (unconfirmed)
Deep model and VAE comparison	Same splits, same threshold procedure, same budget as Phase 1; ablations; several seeds	Metric differences with uncertainty from run-level resampling; inference time and memory vs Phase 1	Only after the upgrade gate (7.1)
Missing/stale data robustness	Inject drops, delay, jitter, stuck-at and clock offsets into replays	Fraction of windows correctly marked DEGRADED/INVALID; zero silent normal outputs on invalid input	Replays with fault injection
Runtime and latency	ROS 2 loopback, then target hardware: window closure → dispatch	p50/p95/p99 latency, deadline misses, CPU/GPU/memory use; model-only and end-to-end reported separately	Mock stream at design rates
Fallback behaviour	Force deadline misses and invalid branch inputs	Correct rung selected; output flagged; measured timing of each rung	Fault-injection runs

10.2 Splits, leakage and statistics

- **Split by complete weld run, workpiece or acquisition session**, never by randomly splitting windows: with a 50 ms window and 10 ms stride, adjacent windows share 80% of their samples, so random splits leak information and inflate scores.
- **Three roles:** training runs (fit), validation runs (threshold, hyper-parameters, feature choices), test runs (evaluated once, after freezing).
- **Coverage:** report performance per operating condition (material, thickness, joint type, speed), not only overall.
- **Uncertainty:** report intervals computed by resampling over runs (not windows) when the number of runs allows it; with few runs, say so.
- **Event-level scoring:** group consecutive flagged windows into events; define detection delay as flagged time minus event onset; define false alarms as flagged events with no reference event, normalized by arc-on hours.
- **Synthetic anomalies** validate software behaviour and delay mechanics; they do **not** establish real-world defect-detection accuracy and are reported separately from results on real data.

11. Risks, dependencies and unresolved cross-team decisions

Each row is classified as a known requirement (**REQ**), proposed design choice (**PROP**), assumption (**ASSUME**) or unresolved decision (**OPEN**). Nothing here has been confirmed by measurement.

Table 19 — *Risks and open questions*

#	Item	Class	Why it matters	Needed decision / mitigation (who)
1	[HIGH] Actual sensor availability and sampling rates	ASSUME	Models depend on streams that may not exist or may differ in rate	Hardware inventory and rate measurement (Team 2, Team 3)
2	[HIGH] Existence of labelled post-weld inspection data	OPEN	No labels → no supervised model or validated quality claim	Identify inspection process and records; define label format (Team 1, Team 3)
3	[HIGH] Definition and source of truth for each prediction target	OPEN	Undefined targets cannot be validated	Document per target (candidate-targets table, Section 6.5); agree with consumers
4	[HIGH] Meaning of the <10 ms requirement	OPEN	Determines architecture (Section 9.1)	Define boundary: model-only, analytics latency or reaction time (Teams 1, 2, 3, 4)
5	[MEDIUM] Window size and stride	PROP	Trades latency, context and overlap; affects leakage	Experiment on replays; agree rate with consumers (Team 3, Team 4)
6	[HIGH] Timestamp source and synchronization hardware	OPEN	Misaligned streams corrupt multimodal features	Choose hardware timestamping/PTP/NTP; measure offsets (Team 2)
7	[HIGH] Approved I/O schema and ROS 2 topic ownership	PROP	Schemas are proposals; vision relay owner unclear	Interface review meeting; freeze v0.1 (all teams)
8	[HIGH] Path-offset units and coordinate frame	PROP	Sign/frame error could drive a wrong correction	Agree `seam_frame` and units (Teams 4, 5, 6)
9	[HIGH] Validated fallback and stale-data behaviour	PROP	Silent failures would look like normal operation	Test in fault-injection stage (Team 3, Team 2)

#	Item	Class	Why it matters	Needed decision / mitigation (who)
10	[MEDIUM] Target edge hardware and measured runtime	OPEN	No timing claim possible without measurements	Name target device; benchmark (Team 3, Team 1)
11	[MEDIUM] Out-of-distribution and changing conditions	ASSUME	Models degrade on unseen parts, materials or settings	Operating-condition tags; re-validation policy
12	[HIGH] Over-reliance on ML output	RISK	Anomaly misread as defect, or advisory acted on prematurely	Shadow mode, advisory-only enumerations, independent safety logic

12. Implementation roadmap and definition of done

The project guide gives a baseline target “within 10 days, targeting 25 October 2026”. Counting from Monday 19 October, 25 October is day 7, so this plan places the **baseline milestone on day 7** and uses days 8–10 (to 28 October) for loopback, integration and review. The schedule needs confirmation by the project lead. Owner roles come from the team structure on the Team 3 overview sheet; names are to be assigned.

Table 20 — *10-day plan*

Day	Date (2026)	Task	Owner role	Depends on	Deliverable
1	Mon 19 Oct	Confirm available signals, rates and clock source with Team 2; circulate schema v0.1 for review; set up repository, environment and experiment tracking	Team Lead; Member 1	Team 2 availability	Signal inventory (confirmed/unconfirmed); schema review request
2	Tue 20 Oct	Mock `/cell/state` publisher at design rates with jitter/drop injection; rosbag2 recording	Member 1; Member 4	Schema v0.1	Publisher node; recorded replay bags
3	Wed 21 Oct	Window builder, synchronization, freshness rules, quality flags; unit tests	Member 1	Day 2	Tested sync module; test report
4	Thu 22 Oct	Feature extraction per Section 6.2; rule baseline R1–R6 (limits as parameters)	Member 3	Day 3; WPS limits	Feature module checked against hand calculations; baseline module
5	Fri 23 Oct	Synthetic anomaly injection and metrics code (P/R/F1, FA/hour, delay); baseline run on replays; **decision: are labels available?**	Member 3; Member 4	Day 4	Metrics report v0 (synthetic only, labelled as such)
6	Sat 24 Oct	Isolation Forest benchmark with run-level splits and threshold procedure; comparison with baseline. If labels exist: LightGBM experiment (XGBoost benchmark optional)	Member 3	Day 5	Benchmark report; supervised report only if labels exist
7	Sun 25 Oct	**Baseline milestone:** end-to-end replay produces schema-valid `process_feedback.json` in shadow mode; ROS 2 node running	Member 4; Team Lead	Days 3–6	Demo + validated JSON outputs
8	Mon 26 Oct	ROS 2 loopback timing: p50/p95/p99, deadline misses; assess ONNX conversion feasibility for chosen models	Member 4; Member 3	Day 7	Latency report (clearly separating the four quantities)
9	Tue 27 Oct	Simulation integration with mock/simulated Team 2 and Team 5 outputs; Phase 2 feasibility memo (data volume, labels, channel rates)	Member 2; Member 3	Teams 2 and 5 mock outputs	Integration log; Go/No-Go memo for Phase 2
10	Wed 28 Oct	Cross-team review; update this report; request schema freeze; definition-of-done check	Team Lead (all)	Days 1–9	Report v1.1; decision log

12.1 Definition of done (baseline)

Done	Criterion
[ ]	Signal inventory confirmed with Team 2; unconfirmed signals marked and unused
[ ]	Schema v0.1 reviewed by Teams 1, 2, 4 and 5; open items recorded
[ ]	Mock publisher and replay produce correctly timed 50 ms windows with exact sample counts
[ ]	Feature formulas unit-tested against hand calculations

Done	Criterion
[ ]	Rule baseline and Isolation Forest evaluated with run-level splits; results reported separately for synthetic and real data
[ ]	Missing/stale-data handling demonstrated: no silent normal output on invalid input
[ ]	`process_feedback.json` messages validate against the schema; unvalidated fields are `null`
[ ]	Loopback latency measured and reported with p50/p95/p99; model-only and end-to-end reported separately
[ ]	Supervised model and Phase 2 work either completed with held-out evidence or explicitly recorded as not started, with reasons
[ ]	All outputs advisory; shadow mode documented; no stop authority in any field

13. References and source basis

13.1 References

Bibliographic details and links were checked against the publisher, arXiv or documentation pages during preparation of this report.

[1]	F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest,” in Proc. 8th IEEE International Conference on Data Mining (ICDM), Pisa, Italy, 2008, pp. 413–422. doi:10.1109/ICDM.2008.17
[2]	scikit-learn documentation, `sklearn.ensemble.IsolationForest`. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html
[3]	G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” Advances in Neural Information Processing Systems 30 (NIPS 2017). https://proceedings.neurips.cc/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf ; code: https://github.com/Microsoft/LightGBM
[4]	T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’16), 2016. doi:10.1145/2939672.2939785 ; arXiv:1603.02754 ; code: https://github.com/dmlc/xgboost
[5]	S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” Advances in Neural Information Processing Systems 30 (NIPS 2017), pp. 4765–4774. arXiv:1705.07874 ; code: https://github.com/slundberg/shap
[6]	S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal and S.-I. Lee, “From local explanations to global understanding with explainable AI for trees,” Nature Machine Intelligence, vol. 2, no. 1, pp. 56–67, 2020. doi:10.1038/s42256-019-0138-9
[7]	D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” arXiv:1312.6114, 2013. https://arxiv.org/abs/1312.6114
[8]	PyTorch documentation, `torch.onnx`. https://docs.pytorch.org/docs/main/onnx.html
[9]	ONNX Runtime documentation, Execution Providers. https://onnxruntime.ai/docs/execution-providers/
[10]	ONNX Runtime documentation, TensorRT Execution Provider. https://onnxruntime.ai/docs/execution-providers/TensorRT-ExecutionProvider.html
[11]	ROS 2 documentation, About Quality of Service settings. https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html
[12]	ROS 2 documentation, rosbag2: Overriding QoS policies for recording and playback (Foxy documentation; page notes that version is end-of-life; concept applies to current rosbag2). https://docs.ros.org/en/foxy/How-To-Guides/Overriding-QoS-Policies-For-Recording-And-Playback.html

Not verified in this report: PyTorch layer documentation for `Conv1d`/`GRU` and the SHAP `TreeExplainer` API page were not retrieved; statements about them rely on references [5], [6] and [8] only.

13.2 Source basis and claim classes

Documents considered: the VENTEK team report guide, the Team 3 master prompt, and the six team overview sheets supplied with the brief. **The individual members’ reports and any hardware documentation were not available**; nothing here claims to summarize them.

Table 21 — *Classes of statements used in this report*

Class	Examples in this report	Evidence
1. Established algorithm / framework facts [FACT]	Isolation Forest isolates by random partitioning; GBDT adds trees fitted to residual errors; TreeExplainer computes exact tree SHAP values; ONNX Runtime uses execution providers	References [1]–[11]
2. Project requirements supplied [REQ]	50 ms window, 1 kHz / 100 Hz / 20 Hz rates, `/cell/state` and `/cell/process_feedback`, <10 ms target, quaternion order, Team responsibilities	Project guide and master prompt
3. Engineering recommendations [REC]	Start with rules + Isolation Forest; run-level splits; score polarity; shadow mode; fallback ladder	Team 3 judgement
4. Assumptions requiring verification [ASSUME]	Uniform 1 kHz sampling; aligned clocks; WPS limits available; labels obtainable	None yet; verification listed in Section 11
5. Results requiring experiments [EXP]	Detection accuracy, latency, benefit of Phase 2, ONNX/TensorRT speed-ups, fallback speed	None — no experiment has been run

13.3 Glossary

Term	Meaning
ZOH	Zero-order hold: repeating the last value until a new sample arrives (here only within freshness limits)
TCP	Tool centre point of the welding torch
WPS	Welding procedure specification: the documented process limits for a weld
GBDT	Gradient-boosted decision trees (LightGBM, XGBoost)
SHAP	SHapley Additive exPlanations: per-prediction feature contributions
VAE	Variational autoencoder; reconstruction error used as anomaly score
NDT	Non-destructive testing
Shadow mode	Outputs are published and logged but not acted upon